Annexe A23 — Le chantier B5 : la face arrière en espace objet

LibraryV3 · Phase G, chantier B · pas B5 · 18 au 20 octobre 2026

Cette annexe historise le pas B5 du chantier de vectorisation. Comme A22, elle se lit comme une leçon : ce qu'on a supposé, ce qu'on a démontré, ce que le prototype a réfuté, ce que le lot a mesuré, et ce qu'il faut retenir. Elle complète l'artefact interactif « Lot B5 pas à pas », qui montre le schéma géométrique, les registres étape par étape et les contre-exemples en direct. Elle répond à une question : **comment savoir qu'une face tourne le dos à la caméra sans lire un seul de ses sommets, et sans jamais contredire le rasterizer ?**

Résultat en une ligne. Sur V1 (32 `sphere_hi`, 508 000 faces), `Render` passe de 1 471 à 1 354 µs (−7,9 %) et l'étage faces de 1 302 à 1 183 µs (−119 µs, −9,1 %), lot appairé du 20 octobre. Les compteurs raster sont **identiques au triangle près** sur les trois scènes, sans une seule correction : 97 % des faces arrière de V1 sont écartées en espace objet, 4 lectures contiguës pour 8 faces. Le levier n'est pas une instruction plus rapide : c'est une identité géométrique qui permet de répondre avant de rassembler les sommets.

1. Le point de départ : un lot arrière coûte autant qu'un lot visible

Après B3 (lot du 18/10), V1 se découpait ainsi, en unités du 16/10 : Render ≈ 1,31 ms = dessin des survivantes ≈ 0,86 ms + tests faces ≈ 0,3 ms + étage sommets ≈ 0,15 ms. Les tests faces sont ceux de B4 : pour 8 triangles, `ClassifyFaces8` rassemble les 3 coins de chaque face (24 chargements de 16 o à des adresses dispersées, 12 `vinsertf128`, 6 unpack, 6 shuffle), puis calcule l'aire signée et la boîte serrée.

Sur un mesh fermé, la moitié des faces tournent le dos à la caméra. Pour elles, tout ce travail aboutit à `front = 0`. Et comme un mesh généré range ses faces anneau par anneau, les 8 faces d'un lot sont voisines : un lot est presque toujours entièrement avant ou entièrement arrière. Seuls les lots de la silhouette sont mixtes. B5 cherche donc à reconnaître un lot arrière **avant** le gather.

À retenir. Avant de vectoriser un test plus vite, demander si on peut répondre plus tôt, avec moins de données. Le gather des sommets est le poste cher de B4 ; la réponse « arrière » n'a pas besoin des sommets projetés.

2. L'idée géométrique : le sens d'un triangle ne dépend que du côté de son plan

Un triangle 1-2-3 vu de devant tourne dans un sens, comme une horloge vue de face ; vu de derrière, dans l'autre, comme l'horloge vue à travers le cadran. Entre les deux, l'œil est dans le plan du triangle : il le voit par la tranche, son aire à l'écran est nulle. Le rasterizer mesure ce sens par le signe de l'aire (`EdgeFunction`). Ce signe dit donc une seule chose : **de quel côté du plan de la face se trouve l'œil**, autrement dit le signe de la distance signée n·œil + d.

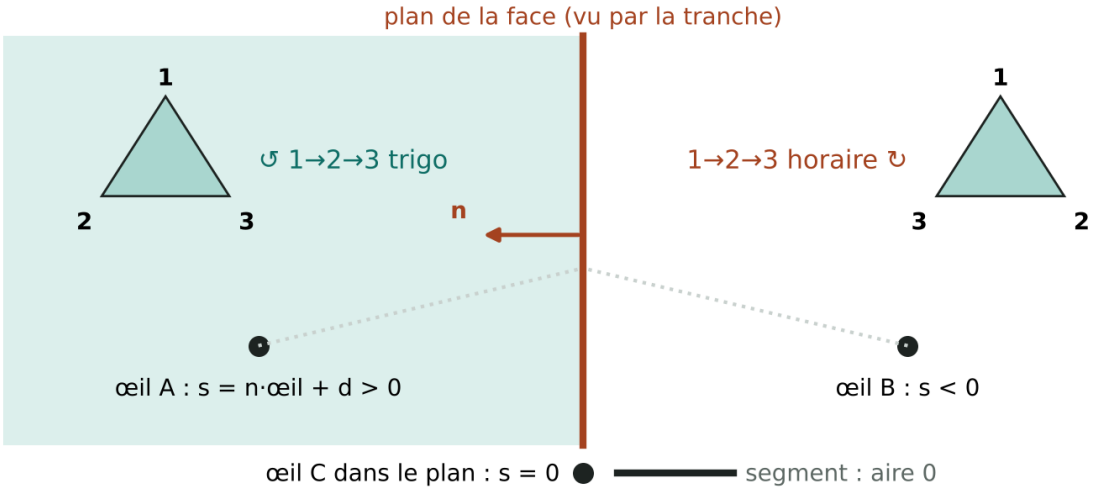


Figure 1 — Le plan de la face vu par la tranche. Les vignettes montrent ce que voit chaque œil du triangle 1-2-3. La moitié teintée est celle de la normale n : la face y est vue.

Ce test ne dépend d'aucun repère. Dans le repère de l'objet, les plans des faces ne bougent jamais : on les calcule une fois au chargement (`BakeFacePlanesSoA`, en double, n normalisée, d = −n·p0), rangés en SoA (4 tableaux alignés 64 o, 16 o par face). Il ne reste qu'un problème : connaître l'œil dans ce repère.

3. La méthode de Scratchapixel, et ses deux angles morts

La méthode classique ramène la position de la caméra dans l'objet : `C_obj = C_monde · inverse(model)`, puis teste n·(C_obj − p0) ≤ 0. Elle est juste et lisible, et son coût (une inverse par instance et par vue) n'est pas le problème. Elle cache deux hypothèses :

- **la caméra a une position finie.** En orthographique, les rayons sont parallèles : il faut une autre formule (n·direction) et une branche. Telle quelle, la formule à position se trompe sur une couronne près de la silhouette (727 faces sur 3 968 dans l'artefact) ;
- **le rasterizer juge comme la géométrie.** Avec une échelle négative, le miroir inverse le sens de rotation de tous les triangles à l'écran. Le test de Scratchapixel donne alors la vérité géométrique, l'écran donne l'inverse : en série dans le même pipeline, le pré-filtre jette une moitié de la sphère, l'écran jette l'autre, **l'objet disparaît**. Il faudrait multiplier par signe(det(model)).

Dans un moteur où le test objet est le seul test, ces deux corrections suffisent. Dans le nôtre, B5 est un raccourci placé devant un test écran qui reste la référence (verrouillé par la TNR, mesuré par les compteurs). Il lui faut la convention du rasterizer **par construction**, pas par une liste de corrections qu'on pourrait oublier.

4. L'œil homogène E, lu directement dans la mvp

Une coordonnée clip est un produit scalaire. MulRow calcule x_clip = x·m[0][0] + y·m[1][0] + z·m[2][0] + 1·m[3][0] : c'est P·c0, avec P = (x, y, z, 1) et c0 la colonne 0 de la mvp. De même y_clip = P·c1, z_clip = P·c2, w_clip = P·c3. Une colonne est une « règle » qui mesure une coordonnée, et l'ensemble des points où elle vaut 0 est un plan : x_clip = 0 (la verticale centrale de l'écran), y_clip = 0 (l'horizontale centrale), w_clip = 0 (le plan de l'œil, parallèle à l'écran). La mvp partant du repère **objet**, ces plans y sont déjà exprimés : aucune inverse.

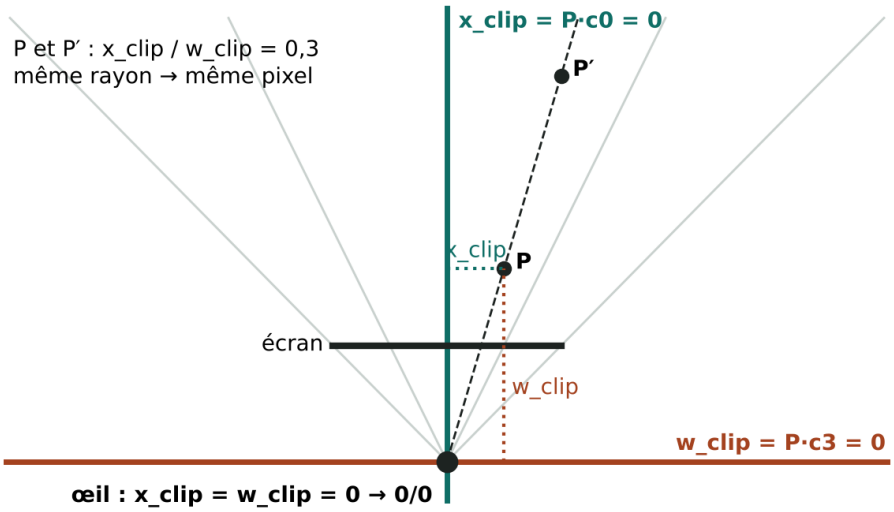


Figure 2 — Vue de dessus, repère de la caméra. Un pixel est le rapport x_clip / w_clip : il est le même pour tous les points d'un rayon (P, P′). L'œil est sur tous les rayons : son rapport vaut 0/0.

L'œil est le seul point qui annule x_clip, y_clip et w_clip, d'où les trois équations E·c0 = E·c1 = E·c3 = 0 : l'intersection de trois plans (figure 3). On laisse c2 de côté pour deux raisons : z ne sert qu'au Z-buffer (le sens à l'écran ne le lit jamais), et l'œil n'annule pas z_clip (en reverse-Z, z_clip = near à l'œil). Avec E·c2 = 0 en plus, la seule solution serait E = 0.

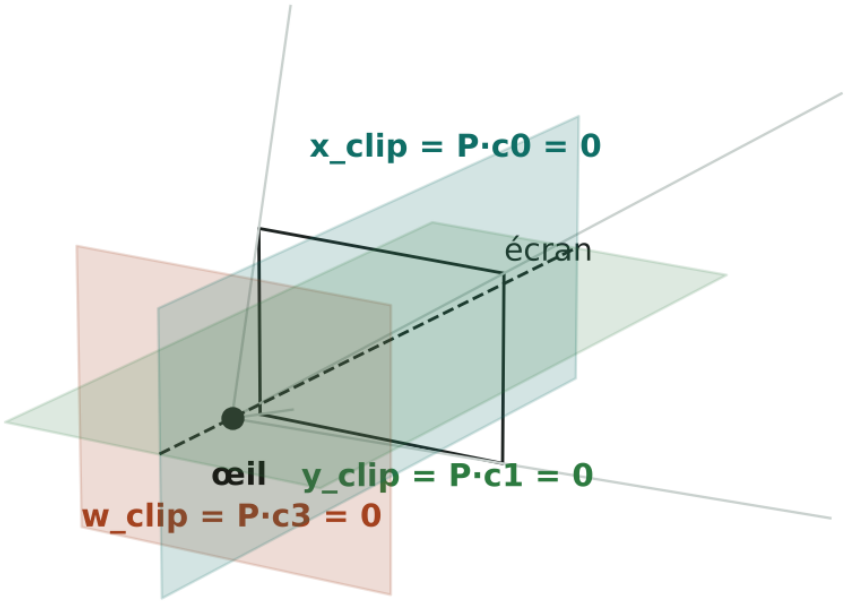


Figure 3 — Les trois plans donnés par les colonnes 0, 1 et 3 de la mvp se coupent en un seul point : l'œil. Dans le repère objet, ils sont simplement déplacés avec la caméra.

Résoudre, à quoi servent les produits vectoriel et mixte. En 3D, b × c est perpendiculaire à b et à c : il résout d'un coup v·b = 0 et v·c = 0. Le produit mixte a·(b × c) est le déterminant det(a, b, c), volume signé du parallélépipède ; il est nul si a est dans le plan de b et c. En 4D, il faut un vecteur perpendiculaire à trois vecteurs : on range c0, c1, c3 dans une matrice C de 4 lignes × 3 colonnes (ses lignes sont les `r_i = (m[i][0], m[i][1], m[i][3])`) et la composante k de E est le cofacteur $(-1)^k$·det(C privée de sa ligne k), chacun calculé par un produit mixte :

```
// HomogeneousEye (FaceStage.h) — 4 déterminants 3x3, aucune inverse, aucune division
E.x =  r1 · (r2 × r3)
E.y = -r0 · (r2 × r3)
E.z =  r0 · (r1 × r3)
E.w = -r0 · (r1 × r2)
// perspective : E = λ·(œil_objet, 1)     ortho : E = λ·(direction vers la caméra, 0)
```

Pourquoi E vérifie les trois équations sans rien résoudre : par construction des cofacteurs, E·v = det[v | c0 | c1 | c3] (développement de Laplace). Pour v = c0, la matrice a deux colonnes égales, donc E·c0 = 0 ; idem pour c1 et c3.

Exemple à la main : caméra de V1 à l'origine, objet poussé de 5 unités devant elle (model = translation (0, 0, −5)). L'œil doit être en (0, 0, +5) dans l'objet.

Étape	Valeurs
lignes de C	r0 = (1,01487 ; 0 ; 0) r1 = (0 ; 1,73205 ; 0) r2 = (0 ; 0 ; −1) r3 = (0 ; 0 ; 5)
produits vectoriels	r2 × r3 = (0 ; 0 ; 0) r1 × r3 = (8,66025 ; 0 ; 0) r1 × r2 = (−1,73205 ; 0 ; 0)
E	(0 ; 0 ; 8,78906 ; 1,75781) → E.xyz / E.w = **(0 ; 0 ; 5)** ✓
vérifications	E·c0 = E·c1 = E·c3 = 0 ✓ ; E·c2 = 0,17578 = near·E.w ≠ 0 (d'où l'exclusion de c2)

5. Pourquoi le signe est forcément celui de l'écran : Cauchy-Binet

L'aire signée de trois points projetés (x/w, y/w) vaut det[[x0 y0 w0], [x1 y1 w1], [x2 y2 w2]] / (w0 w1 w2). Ce déterminant 3 × 3 est celui du produit X = P·C, où P (3 × 4) contient les sommets homogènes et C (4 × 3) les colonnes x, y, w de la mvp. La formule de Cauchy-Binet le découpe en 4 termes : un mineur de P (qui ne dépend que de la face) fois un mineur de C (qui ne dépend que de la caméra). Les mineurs de P sont le plan de la face L = (n_{brut}, d_{brut}) ; ceux de C, l'œil E. Après le flip Y de ToRaster :

```
aire_raster = −K · (L·E) / (w0·w1·w2),    K = (W/2)·(H/2) > 0
chemin Inside : w0, w1, w2 > 0   =>   face AVANT  <=>  s = L·E > 0
```

Vérification sur un vrai triangle de la sphère (artefact, étape 2b, face 2542) :

k	L_k (face)	E_k (caméra)	$L_k \cdot E_k$
0	0,00575	−3,22637	−0,01856
1	−0,00367	1,57594	−0,00579
2	−0,00635	4,32985	−0,02748
3	−0,00930	1,75781	−0,01635
Σ			**−0,06818** = det(X) calculé directement

Aire prédite : −(1536/2)·(900/2)·det / (w0·w1·w2) = 321,02 px² ; aire float du moteur (EdgeFunction sur rasterBuf) : 321,02 px². L normalisé redonne exactement le plan cuit, et E calculé en double redonne E de HomogeneousEye.

À retenir. L'écran et B5 calculent **le même nombre** de deux façons. Ce n'est pas une heuristique qui « marche souvent » : c'est une identité, valable pour toute mvp (perspective, décentrée, orthographique, miroir).

6. Orthographique et miroir sans une ligne de plus

- **Ortho** : w_clip = 1 pour tous les points (c3 = (0, 0, 0, 1)). Aucun point réel n'annule w : la formule donne E.w = 0, un point à l'infini dans la direction commune des rayons. Le test devient n·E.xyz, le bon test ortho, sans branche.

- **Miroir** : les trois équations fixent E à un facteur λ près. La formule des cofacteurs choisit un λ dont le signe suit det(mvp), donc det(model). Avec une échelle négative, λ < 0 et s change de signe, exactement comme l'aire à l'écran.

Dette révélée (hors B5). Avec une échelle négative, l'écran actuel (aire raster < 0 = avant) jette les faces qui regardent la caméra : l'objet est dessiné à l'envers. B5 reproduit ce comportement à l'identique. Aucune scène n'utilise d'échelle négative aujourd'hui. Correction prévue séparément : inverser la convention quand det(model) < 0, sur les deux tests à la fois.

7. Un pré-filtre prudent, pas un remplacement

Le plan précédent annonçait B5 « non bit-exact par nature ». Le prototype hors moteur a déplacé la question. Sur sphere_hi réelle, 20 000 instances (perspective, ortho, miroir), 79 M faces : le signe de s et celui de l'aire float sont opposés sur 99,97 % des faces ; les 27 326 désaccords (0,03 %) sont des faces rasantes ou sous-pixel, d'aire < 10^{-3} px², et l'aire exacte en double donne **toujours** raison à l'objet. Avec une anisotropie poussée jusqu'à $\times 10^{4}$, même constat : 100 % des écarts sont du bruit d'arrondi de l'écran.

D'où la conception retenue : on ne rejette en objet que les faces **sûrement** arrière, et on laisse l'écran trancher pour les autres, comme avant B5.

```
τ = (1/64) · ( |E.xyz| + |E.w| · maxAbsD )     // kObjBackMargin : angle rasant > 0,9°
rejet si s < −τ                                // _CMP_LT_OQ : NaN -> faux, un doute ne rejette jamais
```

|n| = 1 et |d| ≤ maxAbsD : τ est une fraction de la plus grande valeur que s puisse prendre. Les faces plus rasantes restent à `ClassifyFaces8`.

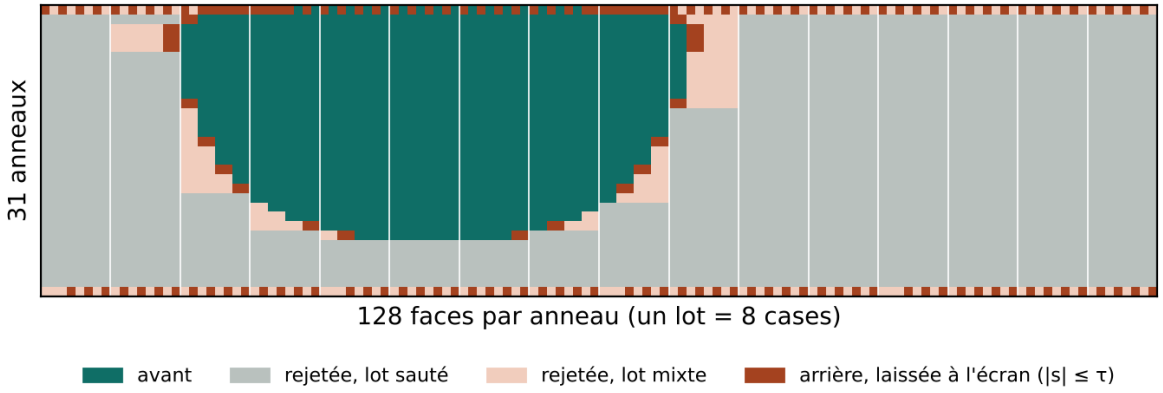


Figure 4 — Carte des 3 968 faces d'une sphère UV (31 anneaux × 128 faces), caméra de V1, calcul float32. 294 lots sur 496 sont sautés sans gather, 60 sont mixtes (silhouette), 142 entièrement avant. Les cases rouges : faces arrière laissées à l'écran parce que |s| ≤ τ.

Contre-exemple : ne sauter que les lots entiers. Si l'on ne masquait les faces rejetées que dans les lots entièrement arrière, une face rejetée en objet serait jetée quand ses 7 voisines le sont aussi, et dessinée sinon (quand l'écran se trompe d'un arrondi). Son sort dépendrait de l'ordre des faces dans le tableau d'indices. D'où le masque `keep = valid & ~objBack`, appliqué à `front` et `survive` dans tous les lots.

À retenir. Un raccourci placé devant une référence ne doit jamais la contredire. On le rend prudent (marge τ, NaN non rejeté) et indépendant du découpage (masque keep) ; la référence garde le dernier mot.

8. L'algorithme, stade par stade, face à Scratchapixel

Stade	Scratchapixel	B5
0 · chargement (une fois)	plans des faces en espace objet	identique : FacePlanesSoA, en double, + maxAbsD
1 · par (instance, vue)	inverse(view) puis inverse(model) ; C_obj : un point	4 produits mixtes sur la mvp ; E : un point homogène ; τ
2 · par face	s = n·(C_obj − p0)	s = ((nx·ex + ny·ey) + nz·ez) + d·ew, sans FMA
3 · décision	s ≤ 0 : jetée ; le test objet décide seul	s < −τ : jetée sans lire ses sommets ; sinon l'écran décide
4 · par lots de 8	—	4 lectures contiguës ; lot entièrement rejeté : lot suivant, ni gather ni test écran
Ortho	2e formule et branche	E.w = 0, même formule
Miroir	contredit l'écran	λ < 0, d'accord avec l'écran
Division	oui (w de l'inverse)	jamais : rien ne déborde quand l'œil part à l'infini

9. Le code

Fichier	Contenu
`Geometry/MeshClass.h/.cpp`	`FacePlanesSoA` (nx, ny, nz, d alignés 64 o, bourrage à zéro, triangles seuls, maxAbsD) ; `BakeFacePlanesSoA()` en double
`Ressources/ResourceManager.cpp`	`RegisterMesh` appelle la cuisson, à côté de `BakePositionsSoA`
`Core/Profiler.h`	compteur `TrisObjRejected`, ajouté en fin d'enum (colonnes des CSV inchangées)
`Rendering/FaceStage.h`	`HomogeneousEye`, `kObjBackMargin`, `ObjBackThreshold`, `ObjBackMask8`, `ExactRasterArea` (vérité en double, Debug)
`Scene/RenderSystem.cpp`	`EmitTriangleList8` : diffusions hors boucle, test objet avant le gather, masque keep ; filet Debug à deux mailles
`Test/Test_B5_ObjBackface.cpp`	test hors moteur (EXE) : SIMD = scalaire, rejet ⇒ arrière exact, identité de signe

```
// ObjBackMask8 : bit k = 1  <=>  face k SUREMENT arrière. Diffusions faites par l'appelant, hors boucle.
const __m256 s = _mm256_add_ps(_mm256_add_ps(_mm256_add_ps(
    _mm256_mul_ps(_mm256_load_ps(nx), ex),          // nx*ex
    _mm256_mul_ps(_mm256_load_ps(ny), ey)),         // + ny*ey
    _mm256_mul_ps(_mm256_load_ps(nz), ez)),         // + nz*ez
    _mm256_mul_ps(_mm256_load_ps(d),  ew));         // + d*ew
return uint32_t(_mm256_movemask_ps(_mm256_cmp_ps(s, negTau, _CMP_LT_OQ)));

// EmitTriangleList8, par lot
const uint32_t objBack = valid & ObjBackMask8(nx + base, ny + base, nz + base, d + base, ex, ey, ez, ew, negTau);
if (objBack == valid) continue;                     // ni gather, ni test écran
const FaceMasks8 m = ClassifyFaces8(rv, idx, vp);   // lot mixte ou avant : B4 inchangé
const uint32_t keep  = valid & ~objBack;
const uint32_t front = m.front & keep;  uint32_t surv = m.survive & keep;
```

10. La validation

Dans l'ordre du chantier B : test hors moteur, TNR, run Debug avec filet, asm Release, lot appairé.

Test hors moteur

Mesure	Résultat
faces, instances	11 907 000 faces, 3 000 instances (perspective, ortho, miroir, échelle non uniforme ×0,5 à ×1,5)
rejetées en objet ; lots sautés	49,7 % ; 43,9 %
désaccords SIMD / scalaire	0
désaccords de signe (|aire| ≥ 1 px²)	0
rejets faux (aire exacte en double « avant »)	0
corrections de l'écran float	257 (aire float « avant », aire exacte « arrière »)

Filet Debug à deux mailles. (1) Dans `EmitTriangleList8`, chaque masque SIMD est refait en scalaire au bit près. (2) Dans `RenderView`, chaque face rejetée en objet est rejugée par son aire raster calculée en double de bout en bout (`ExactRasterArea`), indépendante des deux chemins float ; elle doit être arrière.

Assembleur Release

Le pré-filtre est entièrement inliné. Avant la boucle : 5 `vbroadcastss` (ex, ey, ez, ew, −τ). Dans la boucle : 4 `vmulps` à opérande mémoire (le chargement disparaît dans la multiplication, forme optimale), 3 `vaddps`, `vcmpps ..., 17` (17 = _CMP_LT_OQ), `vmovmskps`, puis `je` vers le lot suivant avant le premier `vinsertf128`. `keep` devient un seul `bzhi`. Aucun `vfmadd` ; seul `call` : `DrawTriangle`, précédé d'un `vzeroupper`.

Deux détails sans action : les diffusions repassent par la pile et sont relues après chaque appel (l'ABI Windows x64 ne préserve pas la moitié haute des ymm) ; les 4 pointeurs `planes.*.data()` sont relus à chaque lot (piste B5d). Coût L1, invisible devant les gathers économisés.

Lot appairé A (B3) / B (B5), 20 octobre

i7-1265U, affinity = 4, secteur, mode performances, ordre A B B A. Compteurs identiques sur les trois scènes, **aucune correction** :

Scène	TrisRasterized (A = B)	TrisEarlyRejected (A = B)	TrisObjRejected (B)
V1	249 466	231 346	250 776 (49,4 % des faces Inside, 97 % des faces arrière)
V2	5 036	12	10 656 (67 %)
L	493	224	468 (47 %)

Run V1	Render	RenderVerts	Étage faces	Clear
A1 (B3)	1 463	168	1 295	455
A2 (B3)	1 478	170	1 309	461
B1 (B5) †	1 365	171	1 194	460
B2 (B5)	1 344	173	1 171	457

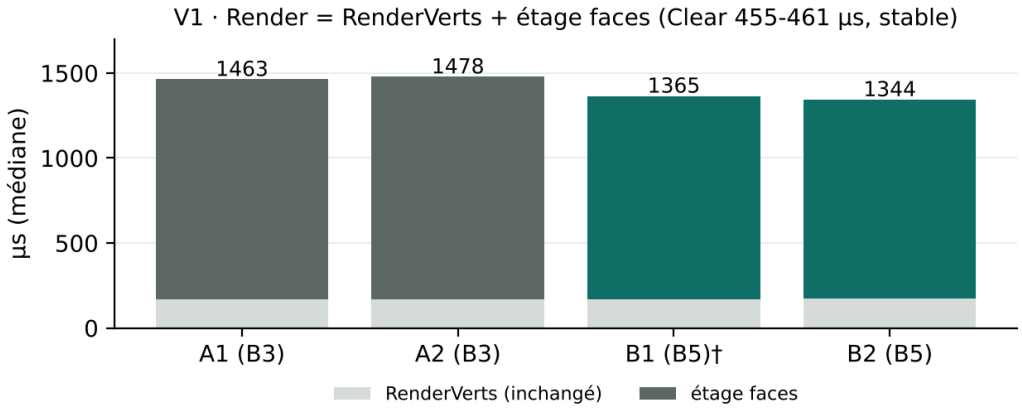


Figure 5 — V1, médianes en µs. RenderVerts et Clear ne bougent pas : les deux thermomètres sont d'accord, aucune correction. † B1 : médiane cohérente, p95 perturbé (4,9 ms) par une charge extérieure.

- **V1** : étage faces −119 µs (−9,1 %), Render −7,9 %. Prédiction : −80 à −150 µs et −6 à −11 %. **Confirmée, au milieu de la fourchette.** Avec B2 seul : étage faces −10 %.
- **V2** : dans le bruit, comme prévu (gain théorique ≈ 10 µs). A1 = 2 060 µs contre ≈ 2 700 µs pour A2, B1, B2, à compteurs identiques et Clear stable : V2 est **bimodal**, déjà constaté au lot B3. Un écart A/B isolé sur V2 ne prouve rien.
- **L** : A1 non recevable (M19 : Animation +17 %, tout le run plus lent). A2, B1, B2 : Render / Clear 0,519 → 0,515 / 0,494, dans le bruit, comme prévu.

11. Ce qu'il faut retenir

- Une face est arrière si l'œil est du mauvais côté de son plan. Ce test se fait dans le repère objet, où les plans sont cuits une fois.
- L'œil homogène E se lit dans la mvp : intersection des plans x_clip = 0, y_clip = 0, w_clip = 0, calculée par 4 produits mixtes. Ni inverse, ni division, ni branche ortho, ni cas miroir.
- Cauchy-Binet fait de l'aire écran et de L·E le même nombre : B5 hérite de la convention du rasterizer par construction.
- Un raccourci devant une référence doit être prudent (τ) et indépendant du découpage (keep). C'est ce qui garde les compteurs bit-exacts.
- Le prototype hors moteur a réfuté l'hypothèse « non bit-exact par nature » du plan : les seuls désaccords étaient des erreurs d'arrondi de l'écran.
- Contre-exemple : C·inverse(model) passe le cas courant (perspective, échelle positive) et casse les deux autres ; un test qui ne couvre que le cas courant la déclare valide.

Et maintenant

Découpage V1 après B5 (unités du 20/10, Render ≈ 1,35 ms) : **dessin des survivantes ≈ 0,96 ms (≈ 71 %)**, tests faces ≈ 0,22 ms (≈ 16 %), étage sommets ≈ 0,17 ms (≈ 13 %). Le dessin des pixels domine V1 et presque tout V2 : c'est le prochain chantier, l'occasion aussi de comprendre la bimodalité de V2. Pistes B5 rangées : B5b (pré-filtre sur le chemin Intersect avant clipping), B5c (ne projeter que les sommets des faces non rejetées), B5d (pointeurs de plans hors de la boucle).